Universidade Federal do Rio de Janeiro

Relatório de Projeto Final COCADA 25.2

Estudo experimental de PCA em eigenfaces

Aluno	Eduardo Tavares Godinho (DRE 124019014)
Professor	João Antônio Récio da Paixão
Disciplina	Computação Científica e Análise de Dados

Rio de Janeiro, 2 de dezembro de 2025

Conteúdo

1 Ideia Inicial **1**

2 Padronização dos Datasets **1**
2.1 Olivetti Faces 1
2.2 Dataset autoral 2

3 Conceitos e Aplicação **5**
3.1 Análise de Componentes Principais (PCA) 5
3.2 Técnica de Eigenfaces 7

4 Experimentos **11**
4.1 Reconstrução 11
4.2 Interpolação 13
4.3 Geração de Face Sintética 15
4.4 Concatenação de Componentes 18

5 Imagens Extras **19**

6 Referências **21**

1 Ideia Inicial

Durante a aula de apresentação de um "rascunho"para o projeto, foram registradas as ideias iniciais para um projeto que explorasse um uso da análise de componentes principais (PCA) no reconhecimento de padrões e individualidades em imagens de rostos, proporcionando a realização de experimentos visualmente interessantes e uma aplicação dos conhecimentos abordados em sala.

Pensando nos requisitos da criação de uma aplicação, as entradas seriam a seleção de uma operação específica (dentre reconstrução, sintetização ou interpolação), além de identificações de rostos de um dataset (junto com especificações de porporção, dependendo da operação selecionada). Já as saídas podem ser reconstruções de posto menor da imagem identificada, imagens interpoladas usando uma combinação linear dos rostos selecionados projetados nos componentes principais do dataset, entre outros.

Por fim, foram levantadas duas possibilidades para bancos de dados a serem usados: o Olivetti faces, dataset de imagens de rostos utilizado para o desenvolvimento e teste de algoritmos de reconhecimento facial; ou um dataset próprio, criado com imagens de amigos padronizadas por mim. Para a realização simplificada do projeto, evitando explosão de dimensões dos dados, a formatação do dado deve ser em grayscale, com tamanho normalizado em 64x64 ou 100x100 (no casos do Olivetti faces e do dataset próprio, respectivamente).

2 Padronização dos Datasets

Como descrito na ideia inicial do projeto, foram consideradas duas opções de banco de dados para a sua realização. Por isso, o código da ferramenta de experimentação foi adaptado para contemplar tanto o uso do dataset Olivetti faces quanto para criar um dataset padronizado com quaisquer fotos de rostos e utilizá-lo para obter eigenfaces e suas derivações. Devido às suas diferenças, diferentes medidas foram tomadas para padronizar cada conjunto de imagens, mencionadas a seguir.

2.1 Olivetti Faces

Por ser um banco de dados extremamente famoso no desenvolvimento de tecnologias de reconhecimento facial, o Olivetti pode ser acessado através de uma biblioteca do Python, em que já vem com imagens em grayscale centralizadas no rosto e com um formato padronizado de 64x64 pixels.

Figura 1: Amostra de algumas imagens do dataset com identificador de cada pessoa

2.2 Dataset autoral

Para um dataset original, por outro lado, seria necessário um algoritmo para formatar todas as imagens, preferencialmente de maneira automática. Depois de coletar (consensualmente) imagens dos rostos de amigos em diferentes expressões faciais (5 fotos por pessoa), foi necessário um algoritmo capaz de:

1. detectar o rosto e o deixar na região central da imagem;
2. converter para grayscale
3. redimensionar para 100x100 (dimensões escolhidas para abranger características como cabelo e formato do queixo)
4. normalizar as intensidades dos pixels para uma escala 0-255

Por isso, o código para padronizar o dataset usa as bibliotecas do Python cv2 e Mediapipe, para trabalhar com arquivos de imagens suas propriedades, além de detectar rostos para um recorte automático e adequado.

```
import cv2
import mediapipe as mp
import numpy as np
import os
```

```python

IMG_SIZE = 100  # padroniza todas para 100x100
INPUT_DIR = "dataset_raw"
OUTPUT_DIR = "dataset_padronizado"

os.makedirs(OUTPUT_DIR, exist_ok=True)

mp_face = mp.solutions.face_detection.FaceDetection(
    model_selection=1, min_detection_confidence=0.5)

def process_image(path, output_path):
    img = cv2.imread(path)
    h, w, _ = img.shape

    # Detecta rostos
    results = mp_face.process(cv2.cvtColor(img, cv2.
    COLOR_BGR2RGB))

    if not results.detections:
        print("Nenhum rosto encontrado em:", path)
        return

    detection = results.detections[0]
    box = detection.location_data.relative_bounding_box

    # Bounding box relativo -> absoluto
    x = int(box.xmin * w)
    y = int(box.ymin * h)
    bw = int(box.width * w)
    bh = int(box.height * h)

    # Ajusta o ROI para evitar cortes ruins
    x = max(0, x - bw//6)
    y = max(0, y - bh//6)
    bw = min(w - x, bw + bw//3)
    bh = min(h - y, bh + bh//3)

    face = img[y:y+bh, x:x+bw]

    # Converte para grayscale
    face_gray = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY)

    # Redimensiona para dimensoes fixas
    face_resized = cv2.resize(face_gray, (IMG_SIZE, IMG_SIZE)
    )

    # Normaliza (0-255)
    face_norm = cv2.equalizeHist(face_resized)

```

```python
    # Salva
    cv2.imwrite(output_path, face_norm)
    print("Salvo:", output_path)

# LOOP PRINCIPAL

for person in os.listdir(INPUT_DIR):
    person_folder = os.path.join(INPUT_DIR, person)
    if not os.path.isdir(person_folder):
        continue

    # pasta de saida
    out_folder = os.path.join(OUTPUT_DIR, person)
    os.makedirs(out_folder, exist_ok=True)

    for filename in os.listdir(person_folder):
        if filename.lower().endswith((".jpg", ".png", ".jpeg")):
            in_path = os.path.join(person_folder, filename)
            out_path = os.path.join(out_folder, filename)

            process_image(in_path, out_path)
```

Figura 2: Pasta com as imagens do dataset autoral padronizadas.

3 Conceitos e Aplicação

3.1 Análise de Componentes Principais (PCA)

Para a realização e compreensão do projeto, o conceito abordado em aulas ao longo do período mais importante de se ter em mente é o de PCA. Ele consiste em, a partir de uma base de dados formada por vetores, encontrar uma matriz de posto menor composta por vetores que melhor "explicam"o dataset usando menos recursos computacionais. Essa matriz é composta pelos componentes principais do dataset, cuja forma de obtenção pode ser demonstrada seguindo o seguinte raciocínio:

Digamos que dispomos de um conjunto qualquer de dados, composto por dois vetores $\vec{a}_1$ e $\vec{a}_2$. Nosso propósito é determinar um vetor $\vec{v}$, de mesma dimensão, que melhor represente simultaneamente esses vetores. Esse seria o primeiro componente principal, que minimiza o erro das projeções de ambos os vetores do dataset nele mesmo. De maneira algébrica, podemos modelar o problema como:

$$\arg\min_{\vec{v},\, k_1,\, k_2} \left\|\vec{a}_1 - k_1\vec{v}\right\|^2 + \left\|\vec{a}_2 - k_2\vec{v}\right\|^2.$$

Achar simultaneamente $\vec{v}$ e os coeficientes escalares k_i tornaria o processo relativamente trabalhoso. Uma alternativa mais conveniente consiste em mudar a forma de pensar o ajuste: em vez de buscar valores k_i que aproximem cada vetor individualmente, procuramos o vetor $\vec{v}$ cuja projeção maximize, no sentido global, o alinhamento dos dados. Para isso, impomos a condição $\|\vec{v}\| = 1$ e buscamos minimizar o erro das projeções. Assim, podemos reformular o problema como:

$$\arg\min_{\|\vec{v}\|=1} \sum_{i=1}^{2} \left\|\vec{a}_i - (\vec{a}_i^T\vec{v})\,\vec{v}\right\|^2.$$

Cada termo do somatório corresponde ao vetor de erro de projeção. Pelo Teorema de Pitágoras, sabemos que:

$$\left\|\vec{a}_i - (\vec{a}_i^T\vec{v})\vec{v}\right\|^2 = \|\vec{a}_i\|^2 - \|\vec{a}_i^T\vec{v}\|^2.$$

Logo,

$$\arg\min_{\|\vec{v}\|=1} \sum_{i=1}^{2} \left(\|\vec{a}_i\|^2 - \|\vec{a}_i^T\vec{v}\|^2\right) = \sum_{i=1}^{2} \|\vec{a}_i\|^2 - \left(\arg\max_{\|\vec{v}\|=1} \sum_{i=1}^{2} \|\vec{a}_i^T\vec{v}\|^2\right).$$

Chegamos então ao problema essencial: maximizar a soma dos coeficientes das projeções dos vetores de dados sobre o vetor $\vec{v}$ (em outras palavras, maximizar a soma das "sombras"formadas pela projeção dos vetores a_i). Desenvolvendo, temos:

$$\sum_{i=1}^{2} \|\vec{a}_i^T \vec{v}\|^2 = \sum_{i=1}^{2} \left(\vec{v}^T \vec{a}_i\right)\left(\vec{a}_i^T \vec{v}\right) = \vec{v}^T \left(\sum_{i=1}^{2} \vec{a}_i\, \vec{a}_i^T\right) \vec{v}.$$

Note que

$$\sum_{i=1}^{2} \vec{a}_i \vec{a}_i^T = AA^T,$$

onde A é a matriz formada pelos vetores coluna $\vec{a}_1$ e $\vec{a}_2$. Assim, o problema de maximização torna-se:

$$\arg\max_{\|\vec{v}\|=1} \vec{v}^T (AA^T) \vec{v}.$$

Como a restrição é $\|\vec{v}\|^2 - 1 = 0$, utilizamos multiplicadores de Lagrange, botando os gradientes das funções como paralelos:

$$\nabla_{\vec{v}}\left(\vec{v}^T (AA^T) \vec{v}\right) = \lambda\, \nabla_{\vec{v}}(\vec{v}^T \vec{v}).$$

Sabendo que

$$\nabla_{\vec{v}}(\vec{v}^T (AA^T) \vec{v}) = 2AA^T \vec{v} \quad \text{e} \quad \nabla_{\vec{v}}(\vec{v}^T \vec{v}) = 2\vec{v},$$

obtemos:

$$2AA^T \vec{v} = 2\lambda \vec{v} \quad \Rightarrow \quad AA^T \vec{v} = \lambda \vec{v}.$$

Descobrimos, portanto, que o vetor que minimiza o erro é exatamente o autovetor associado ao maior autovalor de AA^T, chamada de matriz de covariância. Retomando a expressão do erro total:

$$\sum_{i=1}^{2} \|\vec{a}_i\|^2 - \left(\arg\max_{\|\vec{v}\|=1} \vec{v}^T (AA^T) \vec{v}\right) = \|A\| - \left(\arg\max_{\|\vec{v}\|=1} \|A^T \vec{v}\|^2\right).$$

Chamando de S a função "sombra"que queremos maximizar.

$$S = \arg\max_{\|\vec{v}\|=1} \|A^T \vec{v}\|^2,$$

temos

$$S = \arg\max_{\|\vec{v}\|=1} (A^T\vec{v})^T(A^T\vec{v}) = \arg\max_{\|\vec{v}\|=1} \vec{v}^T(AA^T)\vec{v}.$$

Pela equação dos autovetores, temos:

$$S = \arg\max_{\|\vec{v}\|=1} \vec{v}^T(\lambda\vec{v}) = \arg\max_{\|\vec{v}\|=1} (\lambda\|\vec{v}\|^2) = \arg\max \lambda.$$

Dessa maneira, concluímos que o erro mínimo associado ao PCA é determinado pelo maior autovalor da matriz AA^T, e o vetor que melhor representa os dados é o autovetor correspondente.

Da mesma forma, para obter outros componentes principais (os vetores que melhor explicam a parte que o primeiro componente principal não explicou), podemos aproximar a matriz que explicamos com o primeiro componente principal (projetando esse PC1 nos vetores do conjunto de dados) e subtrair da matriz original.

$$R = A - \vec{v}_1\vec{v}_1^T A$$

Fazendo o mesmo processo de descobrir o componente principal dessa matriz R, encontramos o segundo componente principal da matriz original, que equivale ao autovetor associado ao segundo maior autovalor de AA^T. Isso conta para todos os outros componentes principais de uma matriz A (indo até o n-ésimo sendo n a dimensão de A), mas o importante é representar essa matriz suficientemente bem, usando componentes principais para reduzir a dimensão do dado enquanto mantém a informação desejada.

3.2 Técnica de Eigenfaces

A técnica de Eigenfaces é um método de reconhecimento e análise de faces baseado em ferramentas fundamentais de Álgebra Linear, especialmente autovalores, autovetores e análise de componentes principais. A ideia central é representar rostos como combinações lineares de autovetores de uma matriz de covariância (os componentes principais de um dataset com múltiplas imagens). Esses autovetores — chamados de eigenfaces — capturam padrões essenciais de variação entre diferentes rostos.

O processo começa com a organização das imagens em uma matriz de dados. Cada imagem, originalmente bidimensional, é convertida em um vetor coluna ao ser "achatada" e "esticada"em uma dimensão. Se cada imagem possui $H \times W$ pixels, cada vetor está em $\mathbb{R}^{HW}$. Reunindo n imagens, obtemos uma matriz $X \in \mathbb{R}^{n\times HW}$, onde cada linha representa uma imagem. O primeiro passo matemático é calcular a média de todas as imagens e centralizar o conjunto de dados ao subtrair essa média de cada imagem:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

$$\hat{x}_i = x_i - \mu$$

Agora que o dataset está centralizado, eliminando deslocamentos globais e garantindo que a análise de variância reflita somente diferenças estruturais entre os rostos, podemos encontrar os componentes principais da matriz. Como demonstrado anteriormente, os PCs de uma matriz podem ser obtidos ao calcular os autovetores da matriz de covariância AA^T, em que a ordem de "relevância"desses componentes é determinada pelo tamanho dos autovalores associados a cada autovetor.

No caso do dataset em questão, estamos usando uma matriz com vetores linha representando cada rosto (diferente de como foi visto em sala, com vetores coluna), e por isso precisamos usar a matriz de covariância $C = X^T X \in \mathbb{R}^{HW \times HW}$ (transposta de XX^T), que descreve como os pixels variam conjuntamente entre si. No entanto, como HW costuma ser muito grande (dezenas de milhares de pixels), diagonalizar $X^T X$ diretamente é inviável computacionalmente.

Para contornar esse problema, utilizamos a chamada covariância reduzida, a matriz menor XX^T que vive no espaço das amostras e não dos pixels, portanto não possui o formato de informação que queremos. Apesar disso, XX^T e $X^T X$ compartilham os mesmos autovalores não nulos. Assim, ao diagonalizar XX^T, obtemos autovetores no espaço das amostras, que podem ser projetados de volta para o espaço original das imagens por meio de

$$v_i = X^T u_i,$$

gerando vetores em $\mathbb{R}^{HW}$ que correspondem às direções reais de maior variância no espaço das imagens, os componentes principais do dataset. Isso funciona pois a matriz XX^T representa como combinar as imagens originais para formar um componente principal no espaço dos pixels, ou seja, diz quanto cada imagem contribui para a direção principal que será projetada no "mundo antigo"dos pixels. Esses vetores obtidos nessa transformação, quando normalizados, são as eigenfaces (representando cada pixel em níveis de preto e branco, sendo 1 = preto e 0 = branco).

Para implementar esses métodos para qualquer dataset padronizado no projeto, foram usadas as seguintes funções em python:

```python
def preprocessar(images):

    n, H, W = images.shape

    X = images.reshape(n, H * W)

    media = np.mean(X, axis=0)
    X_centralizado = X - media

    return X_centralizado, media, (H, W)


def calcular_pca(X_centralizado):

    # PCA via covariancia reduzida
    # C = X X^T
    C = np.dot(X_centralizado, X_centralizado.T)

    print("Calculando autovalores/autovetores...")
    autovalores, autovetores_reduzidos = np.linalg.eigh(C)

    # ordenar em ordem decrescente
    idx = np.argsort(autovalores)[::-1]
    autovalores = autovalores[idx]
    autovetores_reduzidos = autovetores_reduzidos[:, idx]

    # converter autovetores reduzidos para eigenfaces
    eigenfaces = np.dot(X_centralizado.T,
   autovetores_reduzidos)

    # normalizacaoo: cada eigenface deve ter norma 1
    eigenfaces = eigenfaces / np.linalg.norm(eigenfaces, axis
   =0)

    return eigenfaces, autovalores
```

Tendo obtido os eigenfaces do dataset escolhido, formamos uma base ortonormal para o subespaço de maior variância do conjunto de rostos. Isso porque os eigenfaces são simultaneamente os vetores que capturam as direções de maior variância dos rostos e os que fornecem a melhor aproximação possível da matriz de imagens em posto reduzido. Apesar de não parecer, essas duas são equivalentes pelo teorema de Eckart–Young, que aborda o problema da análise de componentes principais tanto pela minimização do erro da projeção quanto pela maximização da variância dos dados quando projetados nos PCs.

Uma característica importante é que as eigenfaces correspondem aos padrões mais significativos de variação do dataset: iluminação, formato do rosto, orientação da cabeça, distinção de características, etc. Os autovalores associados a cada eigenface indicam a "quantidade de variância" capturada por aquela direção.

Assim, ordenando os autovetores na ordem decrescente de seus autovalores correspondentes, obtemos as eigenfaces mais relevantes primeiro, transmitindo características mais expressivas de cada pessoa. A soma acumulada dos autovalores permite determinar quantos componentes são suficientes para representar bem as imagens, muitas vezes usando apenas uma fração da dimensionalidade original.

A projeção de um rosto no subespaço das eigenfaces é dada por

$$w = U_k^\top (x - \mu),$$

onde U_k contém as k primeiras eigenfaces como colunas. O vetor w é chamado de assinatura da face, pois codifica a imagem usando poucos coeficientes (o quanto cada componente principal compõe a imagem projetada). A reconstrução aproximada é obtida pela combinação linear das eigenfaces escolhidas:

$$\hat{x} = U_k w + \mu.$$

Quando k é suficientemente grande, a reconstrução é visualmente muito próxima da imagem original; quando k é pequeno, apenas os traços gerais do rosto permanecem. Obs.: vale perceber que estamos trabalhando com um vetor centralizado assim como o resto do dataset, por isso há a subtração da média μ e sua posteiror adição.

Esses procedimentos foram implementados no projeto usando as seguintes funções:

```
def projetar_imagem(img_flat, media, eigenfaces, k):

    # Projeta uma imagem (vetor 1D) no subespaco das k
   eigenfaces.

    img_centered = img_flat - media
    return np.dot(eigenfaces[:, :k].T, img_centered)


def reconstruir_imagem(projecao, media, eigenfaces, k):

    # Reconstroi imagem a partir da projecao nas eigenfaces.

    rec = np.dot(eigenfaces[:, :k], projecao) + media
    return rec
```

4 Experimentos

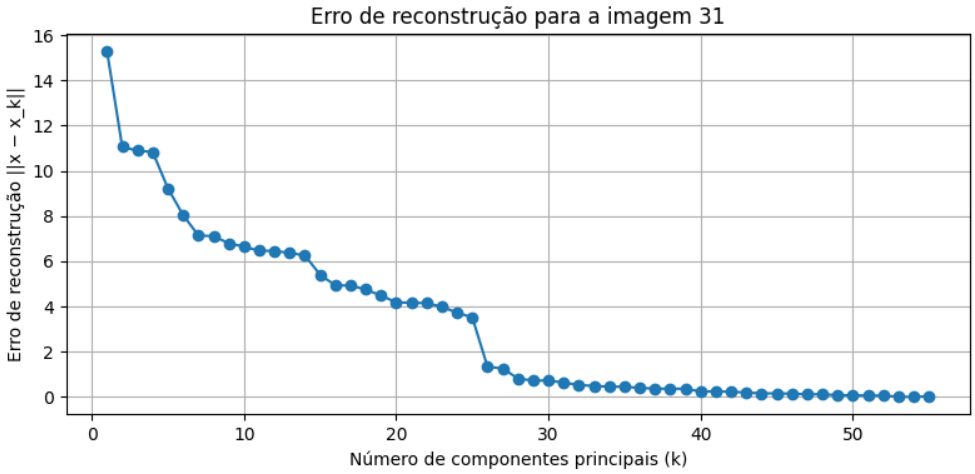
Nesta seção, serão registrados os experimentos possibilitados pela obtenção dos eigenfaces de ambos os datasets, que trazem resultados e entendimentos interessantes sobre a interpretação de rostos e expressões faciais como dados.

4.1 Reconstrução

Como dito anteriormente na seção Conceitos e Aplicação, é possível reconstruir uma imagem usando k eigenfaces do dataset, em que o conteúdo do dado reconstruído é melhor representado quanto mais componentes principais são usados. No entanto, como a intenção dessa reconstrução é economizar recursos, examinamos o k menos custoso que já satisfaz o nível de precisão desejado.

Esse "fitting"se aplica no contexto de rostos ao examinar quantos componentes precisam ser usados para expressar características básicas de uma pessoa, a partir de qual k a expressão facial é bem representada, entre outros aspectos. Por estarmos tratando de dados em formato de imagens, podemos tanto realizar essa examinação por pura observação quanto pela comparação das normas dos vetores originais com suas reconstruções, realizando um gráfico do nível de erro entre elas.

No exemplo abaixo, estão exibidas reconstruções do rosto sorrindo de um dos voluntários para o dataset autoral, juntamente com um gráfico do erro para k componentes utilizados.



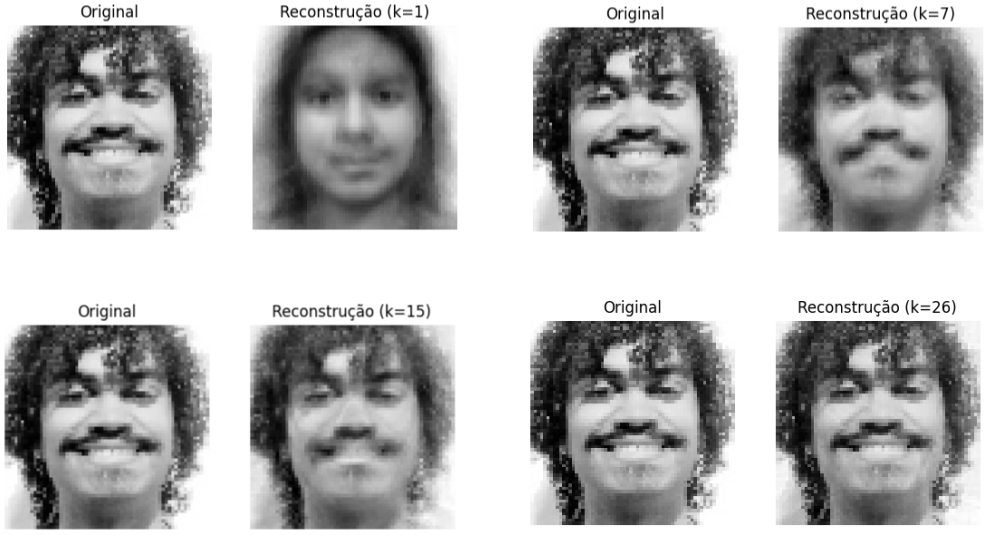


Figura 3: Reconstruções do sorriso de Yuri em 1, 7, 15 e 26 componentes.

Observando o gráfico de erro, percebemos alguns "cotovelos"na curva da linha decrescente: pontos em que há uma grande queda no erro, e em que os poucos k seguintes não contribuem tanto para maior precisão. São os casos, por exemplo, do $k = 7$, que reconstrói as características principais do rosto (avanço enorme em relação ao $k = 1$, que não expressava nada do rosto original). Mesmo assim, essa reconstrução não chega a representar a expressão facial da foto original, mantendo uma face mais neutra ao invés do sorriso. Esse problema é corrigido nas reconstruções de $k = 15$ e $k = 26$, que, respectivamente, capturam uma expressão sorridente um pouco desfocada, e outra reconstrução praticamente perfeita ao olho nu.

A partir desse experimento, podemos demonstrar e observar que os primeiros componentes não são bons em abranger expressões faciais, mas sim caracretísticas físicas (que são aspectos de grande variabilidade entre pessoas, além de serem mais frequentes e por isso mais importantes no dataset).

4.2 Interpolação

Reconstruir rostos em suas características principais e mais simplificadas também nos permite fazer operações entre rostos de maneira mais suave, como a interpolação entre pixels de duas imagens. Esse processo captura singularidades de duas pessoas e as mistura em diferentes proporções de maneira mais visualemtente aceitável do que só sobrepôr imagens completas em pouca opacidade. Isso ocorre pois ambas as reconstruções a serem interpoladas estão projetadas no mesmo espaço vetorial e, além disso, ao usar menos coeficientes, ficam amenizadas e menos distoantes quando misturadas.

A seguir, segue a implementação das funções usadas para essa parte do projeto, além de alguns resultados obtidos e observações interessantes que os acompanham.

```
def interpolar_entre_faces(imgA_flat, imgB_flat, media, eigenfaces, k, n_passos=10):
    """
    Gera n passos de interpolacao entre duas faces, no espaco das eigenfaces.
    """

    # projecoes das imagens
    wA = projetar_imagem(imgA_flat, media, eigenfaces, k)
    wB = projetar_imagem(imgB_flat, media, eigenfaces, k)

    interpolacoes = []

    for i in range(n_passos):
        t = i / (n_passos - 1)
        w_t = (1 - t) * wA + t * wB
        img_t = reconstruir_imagem(w_t, media, eigenfaces, k)
        interpolacoes.append(img_t)

    return np.array(interpolacoes)
```

Os gráficos abaixo mostram diferentes níveis de combinação entre suas reconstruções (sendo t o nível de presença da Imagem B sobre A, como se representasse um progresso de transformação). Para entender como a quantidade de componentes principais usados afetam a interpolação, também foram incluídas iterações usando 10 e 26 componentes, sendo uma suficiente para cobrir as características principais da maioria dos rostos e outra para reconstruir quase perfeitamente todas as faces.

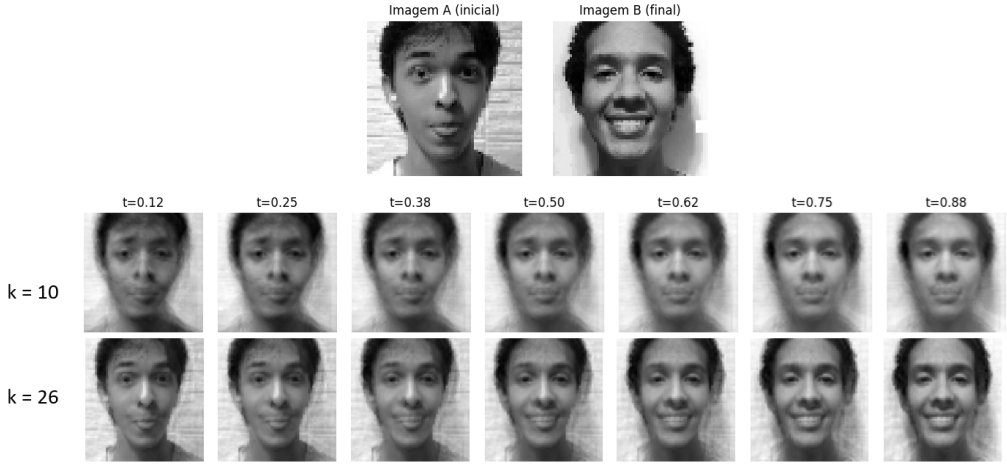


Figura 4: Interpolação entre Arthur (Imagem A) e Alexandre (Imagem B).

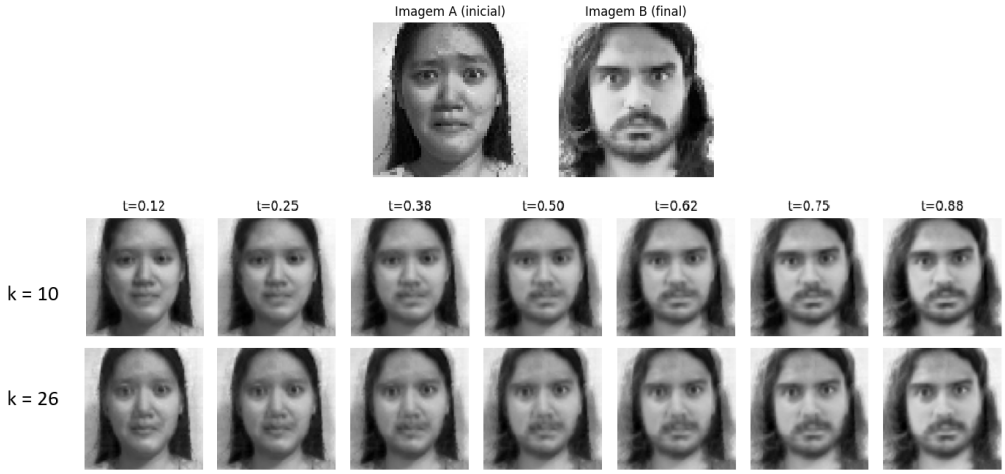


Figura 5: Interpolação entre Ana (Imagem A) e Vinicius (Imagem B).

Assim, podemos notar que os $t = 0.38; 0.50; 0.62$ representam interpolações relativamente convincentes, sendo as de $k = 10$ misturas de reconstruções que ignoram expressões faciais enquanto as com $k = 26$ podem apresentar uma fusão mais explícita das imagens e expressões (na primeira interpolação, por exemplo, a língua da Imagem A se mescla com o lábio inferior da Imagem B).

4.3 Geração de Face Sintética

Nesta seção, descrevemos o papel de cada autovalor no modelo de eigenfaces, explicando como a variância associada a cada componente principal pode ser utilizada para gerar, usando eigenfaces aleatórios, rostos sintéticos minimamente convincentes (que não sejam puro ruído, tendo certa coesão de um rosto).

Considere um conjunto de imagens vetorizadas $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$, e sua média

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

Definimos os dados centralizados como

$$\tilde{x}_i = x_i - \mu.$$

A matriz de covariância é dada por

$$C = \frac{1}{n} \sum_{i=1}^{n} \tilde{x}_i \tilde{x}_i^\top.$$

O PCA resolve o problema espectral

$$C\, u_k = \lambda_k\, u_k,$$

no qual u_k é a k-ésima eigenface e λ_k é o autovalor associado.

Projetando os dados na direção de u_k, obtemos

$$\omega_{ik} = u_k^\top \tilde{x}_i,$$

que representa o coeficiente da imagem x_i na direção da eigenface u_k.

A variância dessas projeções é

$$\mathrm{Var}(\omega_{ik}) = \frac{1}{n} \sum_{i=1}^{n} (u_k^\top \tilde{x}_i)^2.$$

Como u_k é autovetor normalizado, temos:

$$u_k^\top C u_k = \lambda_k,$$

mas também

$$u_k^\top C u_k = \frac{1}{n} \sum_{i=1}^{n} (u_k^\top \tilde{x}_i)^2.$$

Portanto,

$$\boxed{\lambda_k = \mathrm{Var}(\omega_{ik})}.$$

Assim, cada autovalor λ_k mede exatamente a variância real dos dados na direção da eigenface u_k. Eigenfaces são ordenadas de forma decrescente de λ_k, e por isso as primeiras capturam padrões de variação mais fortes, como iluminação, geometria global e contraste facial.

Uma imagem qualquer no subespaço PCA pode ser reconstruída como

$$x \approx \mu + \sum_{k=1}^{K} \omega_k u_k.$$

No contexto de geração de rostos sintéticos, assume-se empiricamente que os coeficientes projetados seguem uma distribuição normal centrada:

$$\omega_k \sim \mathcal{N}(0, \lambda_k),$$

já que a variância real das projeções é precisamente λ_k.

Assim, construímos coeficientes sintéticos amostrando

$$\omega_k = \sqrt{\lambda_k}\, z_k, \qquad z_k \sim \mathcal{N}(0, 1).$$

Substituindo na reconstrução, obtemos o modelo generativo:

$$\boxed{x_{\text{sintético}} = \mu + \sum_{k=1}^{K} \sqrt{\lambda_k}\, z_k\, u_k, \qquad z_k \sim \mathcal{N}(0, 1)}$$

Essa fórmula gera imagens sintéticas coerentes, ao movimentar a imagem média μ ao longo das principais direções de variação dos rostos, com amplitudes proporcionais à variabilidade real observada no dataset. A Tabela **??** na página seguinte descreve cada elemento da fórmula generativa.

Na expressão, u_k equivale à k-ésima eigenface; autovetor correspondente a uma direção de variação facial; $\sqrt{\lambda_k}$ representa o desvio-padrão da distribuição das projeções na direção (já que λ representa a variância dos rostos na direção do autovetor correspondente u_k); z_k é a variável aleatória $z_k \sim \mathcal{N}(0, 1)$, usada para gerar coeficientes sintéticos. Por fim, K equivale ao número de componentes principais utilizados na síntese.

Sua implementação foi relativamente simples, usando a seguinte função e obtendo os seguintes resultados

```
def gerar_face_sintetica(media, eigenfaces, autovalores, k):

    # Cada componente amostrado como:
    # alpha_i ~ Normal(0, sqrt(lambda_i))

    # variancia para cada componente
    desvios = np.sqrt(autovalores[:k])

    # amostra aleatoria nas k dimensoes principais
    coeficientes = np.random.randn(k) * desvios

    # sintetiza a nova imagem
    face_flat = np.dot(eigenfaces[:, :k], coeficientes) +
   media

return face_flat
```

Figura 6: Diferentes imagens sintéticas usando 20 componentes principais.

Observando as imagens geradas, percebemos que, apesar de causar iluminações aleatórias e muitas vezes disformes, há sempre a presença de elementos faciais e expressões distinguíveis, além de acessórios como óculos na maioria das vezes.

4.4 Concatenação de Componentes

Esta seção contempla um experimento derivado da interpolação, idealizado a partir da reflexão sobre quais características dos rostos são capturadas por quais eigenfaces. Ele consiste em somar a reconstrução de uma imagem A usando os $k/2$ primeiros componentes principais com a reconstrução de uma outra imagem B usando o $k/2 + 1$-ésimo até o k-ésimo componente principal (considerando um k par). Dessa maneira, serão representados as características principais da imagem A (fisionomia do rosto, cabelo) mescladas com partes mais difíceis de reconstruir da imagem B (expressão da boca, direção dos olhos). A implementação dessa concatenação de vetores é mostrada a seguir:

```
# interpolacao diferenciada (somando a imagem A projetada nos PCA 1 ate k/2 com imagem B nos PCA k/2 ate k)

# projetar 1 imagem
A_id = 6
wA = projetar_imagem(X_flat[A_id], media, eigenfaces, round(k/2))
wA2 = np.pad(wA, (0,round(k/2)), mode='constant', constant_values=0)

B_id = 0
wB1 = projetar_imagem(X_flat[B_id], media, eigenfaces, k)
wB2 = projetar_imagem(X_flat[B_id], media, eigenfaces, round(k/2))
wB22 = np.pad(wB2, (0,round(k/2)), mode='constant', constant_values=0)
wB = wB1 - wB22

w = wA2 + wB

# reconstruir
rec = reconstruir_imagem(w, media, eigenfaces, k)
rec_img = rec.reshape(shape)
```

Por ser um experimento que parte de um conceito menos exato do PCA (já que partes específicas de certos rostos podem ser mais fáceis ou difíceis de representar usando menos eigenfaces) algumas vezes não há mescla aparente de características. Entretanto, como é mostrado nos resultados abaixo, há a possibilidade de transferir aspectos como expressões faciais sorridentes para rostos neutros e visse-versa, mantendo as características principais do rosto original.

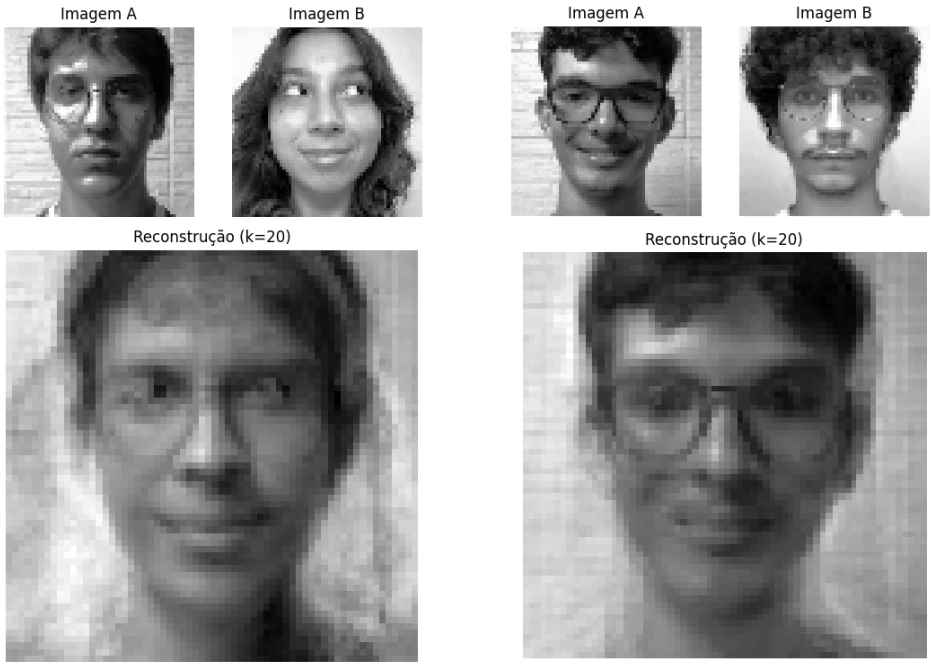


Figura 7: À esquerda, um rosto neutro mesclado com uma expressão sorridente olhando pro lado. À direita, um rosto sorridente mesclado com expressão neutra.

5 Imagens Extras

Nesta seção extra, serão exibidas imagens referentes aos dados da seção 3 do relatório, "Conceitos e Aplicação". Tanto a face média quanto os primeiros 50 eigenfaces de ambos os datasets abordados podem nos trazer observações interessantes.

Repare, por exemplo, que os 10 primeiros eigenfaces do dataset autoral possuem expressões neutras, explicando a limitação de representação e expressando um aspecto geral do dataset. Também é possível perceber uma semelhança entre as áreas de foco entre as faces médias (mean faces) dos datasets, explicada pela maior liberdade da boca em diferentes expressões.

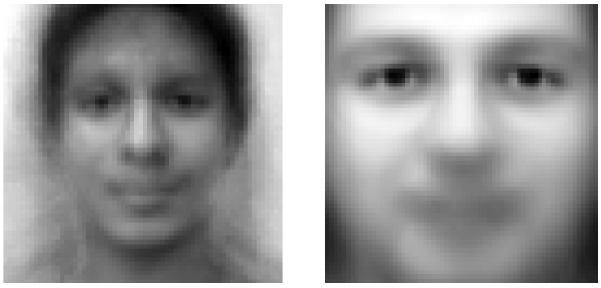


Figura 8: À esquerda, a face média do dataset autoral. À direita, a face média do dataset Olivetti faces.

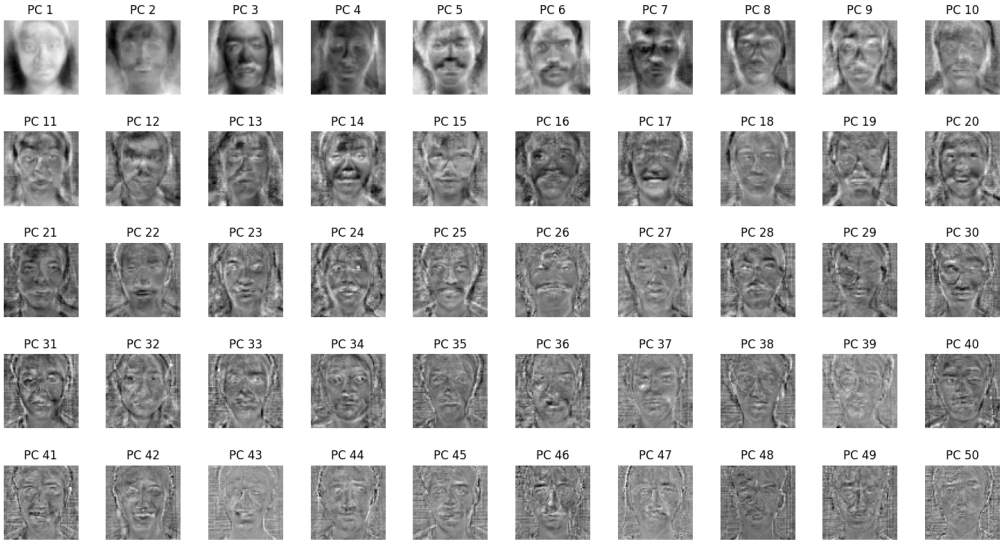


Figura 9: 50 primeiros eigenfaces do dataset autoral.

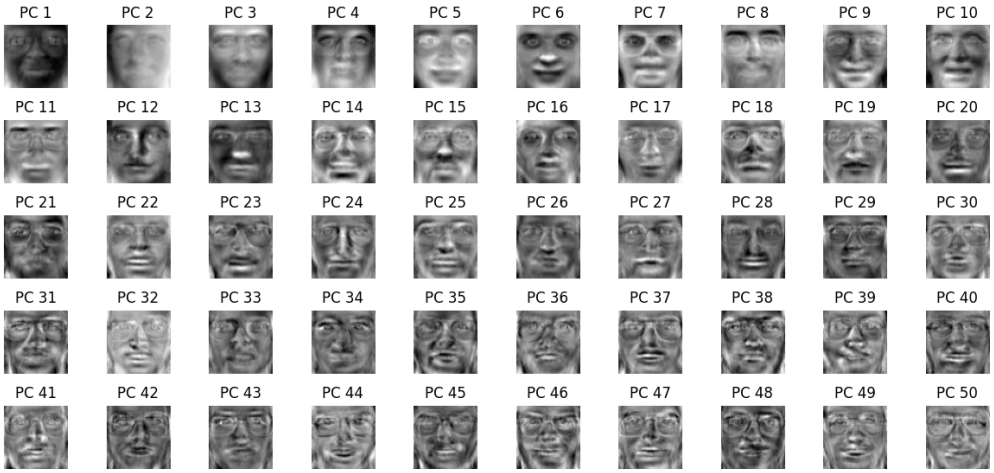


Figura 10: 50 primeiros eigenfaces do Olivetti faces.

6 Referências

- https://www.kaggle.com/code/serkanpeldek/face-recognition-on-olivetti-dataset
- http://stratigrafia.org/8370/handouts/pcaTutorial.pdf
- https://www.datacamp.com/pt/tutorial/covariance
- https://www.visiondummy.com/2014/04/geometric-interpretation-covariance-matrix
- https://github.com/google-ai-edge/mediapipe/blob/master/docs